Ders 11 Genişletilmiş Notu: Ağaçlar, İkili Ağaç Gezintileri ve İkili Arama Ağacı

Ders 11 Genişletilmiş Notu: Ağaçlar, İkili Ağaç Gezintileri ve İkili Arama Ağacı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ağaç Veri Yapısının Tanımı
- 2. Günlük Hayattan Ağaç Örnekleri
- 3. Ağaçların Derecesine Göre Adlandırılması ve Serbest Ağaç
- 4. Ağaç Terminolojisi
- 5. Ağacın Dizi ile Gösterimi
- 6. Ağacın Bağlantılı Liste ile Gösterimi
- 7. İkili Ağaçlar ve Özel Türleri
- 8. İkili Ağaca Eleman Ekleme ve Silme
- 9. Ağaçta Gezinti
- 10. Ağaç Kopyalama ve İki Ağacın Eşitliği
- 11. Çok Çocuklu Ağaçların İkili Ağaç ile Gösterimi
- 12. İkili Arama Ağacı
- 13. Dengeleme ve Dengeli Ağaçlar

1. Ağaç Veri Yapısının Tanımı

Hoca bu dersin konusunu, pek çok algoritmada kullanılan, kendine ait algoritmaları da olan ve günlük hayatta farkında olmadan çok kullandığımız bir veri yapısı olarak tanıttı: ağaç (tree).

Hoca iki tanım verdi. Birincisi kısa ve okuması kolay olan tanımdır:

- Hiyerarşik yapıda birbirine bağlı ve kapalı çevrim (cycle) oluşturmayan **sonlu** düğüm kümesine ağaç denir. Hoca “sonlu” kelimesinin altını özellikle çizdi.

Hoca ayrıca ağacın, ileride göreceğimiz graf veri yapısının özel bir hâli olduğunu söyledi. Graf konusu dönemin son haftalarında işlenecek.

İkinci tanım, ağacı özellikleri üzerinden verir. Aralarında bağlantılar bulunan ve aşağıdaki özellikleri taşıyan düğümler topluluğu bir ağaçtır:

- Kendine doğru kenar içermeyen ve kök (root) olarak adlandırılan özel bir düğüm vardır.
- Kök dışındaki her düğüme tek bir kenar gelir.
- Kökten her düğüme giden tek bir yol vardır.

Hoca bu tanımı tahtada çizerek açıkladı. Kökten aşağı doğru bir düğümden üç, iki ya da tek dal çıkabilir. Bu serbesttir. Fakat alttaki her düğüme yalnız bir yoldan gelinebilir. Hoca çizimdeki iki düğümü kalın bir çizgiyle birbirine bağladığında dört düğümün bir kapalı çevrim oluşturduğunu gösterdi. Böyle bir yapı artık ağaç değildir. Aynı şekilde kökten başlayıp “önce sola sonra sağa” ya da “önce sağa sonra sola” giderek aynı düğüme varılabiliyorsa, iki farklı yol vardır, dolayısıyla çevrim vardır ve yapı ağaç olamaz.

Hocaya göre birinci tanım çok daha basittir. İkinci tanım, “kapalı çevrim içermeme” ve “tek yolla erişilme” gibi kavramları ayrı ayrı açtığı için daha uzundur.

2. Günlük Hayattan Ağaç Örnekleri

Hoca ağacın yalnız bir veri yapısı değil, aynı zamanda çok kullandığımız bir veri gösterim şekli olduğunu vurguladı ve şu örnekleri verdi:

- **Soy ağacı:** Adında bile “ağaç” geçer. Anneanne ve dede evlenir, çocukları olur. Anneniz babanızla evlenir, siz ve kardeşiniz doğarsınız. Annenizin kardeşi (teyze ya da dayı) annenizle aynı seviyededir. Onların çocukları olan kuzenleriniz de sizinle aynı seviyededir. Böylece bir hiyerarşi tanımlanır.
- **Organizasyon şeması:** Yönetim kurulu, CEO, genel müdür, satış ve finans müdürleri, onların altında birim yöneticileri. Tek bir kök vardır ve aşağı doğru bazen bir, bazen üç, bazen beş dallanma olur. Sonunda her şey tek bir yere bağlanır.
- **Tenis turnuvası:** Her maçta iki sporcu karşılaşır ve kazanan bir üst tura çıkar. Ağacın kökü kupayı kazanan sporcudur. Finalde 2, yarı finallerde 4, çeyrek finallerde 8 kişi oynar. Dördüncü turda 16 kişi kalır, öncesinde 32, 64 ve 128. Standart bir turnuvada 128 kişiyle genellikle 4 tur, ardından çeyrek final, yarı final ve final oynanır.
- **Kitabın içindekiler tablosu:** “1. Giriş, 1.1 Literatür taraması, 1.2 Tezin amacı” gibi. Hoca proje raporu şablonlarında da içeriğin aslında bir ağaçla oluşturulduğunu söyledi.
- **Dosya sistemi:** Bir kökten (root) başladığı için doğrudan bir ağaçtır.
- **Veri tabanı indeksleri:** Tek bir kökten başlayıp dallanarak indeks oluşturan veri yapılarıdır.

3. Ağaçların Derecesine Göre Adlandırılması ve Serbest Ağaç

Ağaçlar genellikle derecelerine göre adlandırılır. Her düğümden en fazla iki dal çıkıyorsa, yani her düğümün en fazla iki çocuğu varsa buna **ikili ağaç (binary tree)** denir. Hoca çizdiği örnekte kökü, tek bir yaprağı olan bir tarafı, iki çocuklu iç düğümleri ve iki yaprakla biten dalları gösterdi. Çocuğu olmayan bu düğümlere **dış düğüm (external node)** ya da **yaprak düğüm (leaf node)** denir.

Bir düğümden en fazla üç çocuk çıkıyorsa buna **üçlü ağaç (ternary tree)** denir.

Hoca ardından düğümlerden çıkan dal sayısının 3, 2, 1, 1, 2, 4, 3, 2 gibi sürekli değiştiği bir ağaç gösterdi ve bunu “bir kökü olan ağaç” olarak adlandırdı.

Soru: “En fazla” ifadesini kullandınız. Üçlü ağaçta bir düğümün yalnız iki çocuğu olsaydı, yine üçlü ağaç mı diyecektik?

Cevap: Evet. Bir düğümden en fazla kaç çocuk çıktığı ağacın kaçlı olduğunu belirler. Hoca örnek ağaçta 3, 2, 1 çocuklu düğümlerin yanında 4 çocuklu bir düğüm olduğunu gösterdi ve bu ağacın 4’lü ağaç olduğunu söyledi. Yani en büyük değer ağacın 2’li mi, 3’lü mü, 4’lü mü olduğunu söyler.

> **Ders sonrası not:** Hoca bu açıklamada “bu ağacın seviyesi 4, yani 4’lü ağaç” dedi. Burada kastedilen seviye değil derecedir (degree): bir düğümün sahip olabileceği en fazla çocuk sayısı 4’tür. Seviye, düğümün kökten uzaklığıyla ilgili ayrı bir kavramdır (Bölüm 4).

Hoca bir de kökü belirtilmemiş bir yapı gösterdi. Bu yapı birbirine bağlı düğümlerden oluşur ve kapalı çevrim içermez, yani ağaç şartlarını sağlar. Ama hangi düğümün kök olduğu belli değildir. İstediğiniz düğümü kök seçebilirsiniz. Buna **serbest ağaç (free tree)** denir. Bir düğümü kök olarak seçtiğiniz anda kök, ata ve çocuk hiyerarşisi oluşur. Bu hiyerarşi oluştuktan sonra ağacın kaçıncı dereceden olduğunu da söyleyebilirsiniz.

4. Ağaç Terminolojisi

Hoca, farkında olmadan kullandığı “seviye”, “yükseklik”, “derinlik”, “kardeş” gibi kelimelerin ağacın terminolojisini oluşturduğunu söyledi ve bunları tek tek tanımladı.

4.1 Düğüm ve Kök

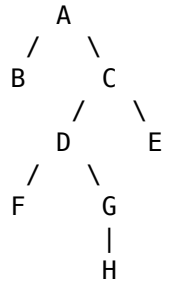
Düğüm (node): İçinde bir veri tutan (en basitinden bir tamsayı) ve diğer düğümlere bağlantılar içeren atomik veri yapısıdır. Genellikle bir `struct` olarak modellenir. Örnekteki A, B, C düğümleri birer karakter tutar. Diğer düğümlere bağlantılar çizgilerle gösterilir.

Kök düğüm (root): Bütün düğümlerin atası olan düğümdür. Hoca, hangi düğümün kök olacağının aslında bize kalmış olduğunu söyledi. D'yi ya da E'yi de kök seçebilirdi. Ama görsel olarak anlamayı kolaylaştırmak için kök genellikle en tepeye çizilir.

Hoca burada bir benzetme yaptı: Doğadaki ağaçların kökü aşağıda, dalları yukarıdadır. Biz bilgisayar mühendisleri ise "ters adamlarız", ağacı da ters çizeriz. Kök tepede, yapraklar aşağıda olacak şekilde ağacı tepe taklak çizmeyi tercih ederiz. Kök her şeyin atasıdır, başlangıç ve erişim noktasıdır. Ağacı nasıl çizdiğinizin hiçbir önemi yoktur. Bu gösterim yalnız okumayı, görmeyi ve anlamayı kolaylaştırır.

4.2 Ebeveyn, Çocuk ve Yaprak

Hocanın terminoloji için kullandığı örnek ağaç şudur:



- **Ebeveyn (parent):** Çocuk düğümleri, yani altında alt ağaçları olan düğümdür. Ata ya da ebeveyn düğüm de denir. Hoca "ebeveyn" kelimesinin Türkçe olmadığını, Türkçede "ana baba" dendiğini de belirtti. Örnekte A hem kök hem ebeveyndir, çünkü solda ve sağda birer alt ağacı vardır. Hoca tek bir düğümün de ağaç olabileceğini hatırlattı.
- **Yaprak (leaf):** Çocuğu, yani alt soyu olmayan düğümdür. Örnekte B bir yapraktır, ebeveyn değildir. E, F ve H de yapraktır.
- **Çocuk (child):** Bir ebeveyne bağlı düğümdür. Sıfır ya da daha çok çocuk içeren düğümlere de çocuk ya da evlat düğüm denir. C, altındaki D, E, F, G, H düğümlerinin atasıdır, ama aynı zamanda A'nın çocuğudur. Ebeveyn olmak, çocuk olma gerçeğini değiştirmez. D de hem ebeveyndir hem C'nin çocuğudur, aynı zamanda A'nın torunudur.

Hoca yaprakların ileride çocuk sahibi olabileceğini de ekledi. Ağaca yeni elemanlar eklendikçe bir yaprak ebeveyn hâline gelebilir. Bu, veri yapısında ne kadar eleman tuttuğunuza ve nasıl bir hiyerarşi kurduğunuza bağlıdır.

4.3 Kardeş

Kardeş (sibling): Aynı ebeveynin çocuklarıdır. Örnekte B ve C, A'nın çocuklarıdır. D ve E, C'nin çocuklarıdır. F ve G, D'nin çocuklarıdır. Bunlar birbirinin kardeşidir.

4.4 Derece, Yükseklik, Derinlik ve Seviye

Ağacın hiyerarşik derinliğini ve çoğulluğunu göstermek için bazı terimler kullanılır.

- **Derece (degree):** Bir düğümün doğrudan sahip olduğu çocuk sayısıdır. Örnekte A, C ve D'nin iki çocuğu, G'nin bir çocuğu vardır. G'nin derecesi 1'dir, D'nin derecesi 2'dir. Ağacın derecesi, düğüm derecelerinin en büyüğüdür. Bu ağacın derecesi 2'dir. Hocanın "ikili ağaç, üçlü ağaç" derken kastettiği de budur.
- **Yükseklik (height):** Kök ile yapraklar arasında var olan en uzun yoldaki toplam kenar sayısıdır. Örnekte kökten en alttaki yaprağa (H) gitmek için A → C → D → G → H yolunda 4 kenar geçilir. Bu yüzden yükseklik 4'tür.

- **Derinlik (depth):** Bir düğümden köke doğru yukarı çıkarken geçilen kenar sayısıdır. Kökün derinliği 0'dır. H'den köke çıkarken 4 kenar geçilir. Bu yüzden H düğümünün derinliği 4'tür. Hoca bunu "ağacın derinliği 4'tür" diye de okudu.
- **Seviye (level):** Kök birinci seviyedir. Kökün çocukları (kardeşler) ikinci seviye, onların çocukları üçüncü seviye, sonrakiler dördüncü ve beşinci seviyedir. Yani seviye = derinlik + 1'dir. Kökün derinliği 0 iken seviyesi 1'den başlar.

  Ders sonrası not: Hoca seviyeyi önce "bir düğümle kök düğüm arasındaki kenar sayısı" diye tanımladı, hemen ardından seviyenin derinlik + 1 olduğunu söyledi. Kenar sayısı tanımı derinliği verir. Derste kullanılan ilişkiye göre seviye, kökten o düğüme kadar olan düğüm sayısıdır (kenar sayısı + 1).

Hoca her düğümün derinliğinin ve yüksekliğinin ayrı ayrı konuşulabileceğini, ağacın yüksekliği ve derinliği için de bu değerlerin en büyüğünün alınacağını söyledi.

Hoca yükseklikte sıfır ile bir değerlerinin karışabileceğini de belirtti: "Eğer çocuğu varsa kökün yüksekliğini bir olarak da kabul edebiliyoruz."

4.5 Ata, Torun ve Alt Ağaç

- A'dan B'ye (aşağı doğru) gidilebiliyorsa A, B'nin atasıdır (ancestor). B de A'nın torunu, alt soyudur (descendant).
- Bir düğüm, bütün alt soyuyla birlikte **alt ağaç (subtree)** olarak adlandırılır. Hoca örnekte A'nın C çocuğuna ait alt ağacı ve C'nin D çocuğuna ait alt ağacı çizerek gösterdi.
- Her ağaç, yapraklara kadar alt ağaçlardan oluşur diye özyinelemeli bir tanım bile yapılabilir.
- Hiç çocuğu yoksa yapraktır. Hiç atası yoksa köktür.

4.6 Derinlik ile Yükseklik Arasındaki İlişki

Hoca derinlik ile yüksekliğin sık karıştırıldığını söyledi ve ikisini başka bir ağaç üzerinde, düğüm düğüm karşılaştırdı:

- **Kök:** Derinliği 0'dır. En uzaktaki yaprağa 3 kenar geçerek gidildiği için yüksekliği 3'tür.
- **Kökün birinci çocuğu:** Köke tek kenarla erişildiği için derinliği 1'dir. Altında bir kenar uzaklıkta bir yaprak olduğu için yüksekliği 1'dir.
- **Kökün ikinci çocuğu (birincinin kardeşi):** Derinliği yine 1'dir. Ama en derindeki yaprağa iki kenar uzaklıkta olduğu için yüksekliği 2'dir.
- **İkinci seviyedeki düğümler:** Hepsinin derinliği 2'dir. İkisi yaprak olduğu için yükseklikleri 0'dır. Birinin altında bir kenar uzaklıkta bir yaprak olduğu için yüksekliği 1'dir.
- **En alttaki yaprak:** Köke üç kenar geçerek ulaşıldığı için derinliği 3'tür, yüksekliği 0'dır.

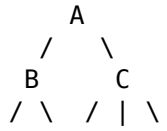
Bu örnekten çıkan sonuç şudur: Kardeşlerin kökle ilişkisi aynı olduğu için **derinlikleri aynıdır**. Ama yükseklikleri çocuklarına, yani alt soylarına bağlı olduğu için **farklı olabilir**. Bir atanın yüksekliği de daha yüksek olan çocuğu üzerinden hesaplanır. Ağacın genelinden bahsederken ağacın yüksekliği kökün yüksekliğinden, ağacın derinliği de en derindeki yaprağın derinliğinden okunur.

Hoca bu örnekte "Eğer hiçbir çocuğu olmasaydı, sadece tek bir kök olsaydı, bunun yüksekliği $h = 1$ diyorduk" dedi.

> **Ders sonrası not:** Hoca yüksekliği kenar sayısıyla tanımladı ve yaprağın yüksekliğini 0 verdi, ama tek düğümlü ağacın yüksekliğini 1 aldı. Kenar sayısıyla tanımda tek düğümlü ağacın (yani yaprağın) yüksekliği 0'dır. Yüksekliği düğüm sayısıyla tanımlayan kaynaklarda ise 1'dir. Hangi tanımın kullanıldığı her zaman belirtilmelidir.

4.7 İkinci Terminoloji Örneği

Hoca terminolojiyi bir ağaç üzerinde toplu olarak tekrar etti:



```
D  E F  G  H
           |
           I
```

- A düğümü köktür.
- B düğümü, D ve E düğümlerinin ebeveynidir.
- C düğümü A'nın çocuğudur, aynı zamanda B'nin kardeşidir. İkisi de ikinci seviyededir.
- D ve E düğümleri B'nin çocukları, A'nın torunlarıdır.
- Slaytta yeşille gösterilen D, E, F, G ve I düğümleri dış düğüm (external node) ya da yaprak düğümdür.
- A, B, C ve H hem ebeveyn düğümdür hem de iç düğüm (internal node) olarak adlandırılır, çünkü çocukları vardır.
- E'nin seviyesi 3'tür: kök birinci seviye, çocukları ikinci seviye, E kökün torunu olduğu için üçüncü seviyededir.
- Ağacın yüksekliği 3'tür. En uzaktaki yaprak I'dır ve A → C → H → I yolunda 3 kenar geçilir. Hoca bunun derinlik olarak da okunabileceğini söyledi: I'nın derinliği de 3'tür.
- B'nin derecesi 2'dir. Ağacın derecesi 3'tür: düğümlerden 2, 2, 3 ve 1 çocuk çıkar, en büyüğü C'nin 3 çocuğudur.
- I'nın ataları, yukarı doğru köke kadar giderken geçilen H, C ve A'dır. Hoca bunu "babası, dedesi, büyük dedesi" diye okudu.
- C'nin alt soyu F, G, H ve I düğümlerinden oluşur.

5. Ağacın Dizi ile Gösterimi

Hoca ağacın nasıl oluşturulacağı sorusuna geçti. Ağaç derecesine göre oluşturulmalıdır: ikili, üçlü, beşli, onlu ağaç olabilir. Ama hoca ileride bütün ağaçların ikili ağaç olarak modellenebileceğini göstereceğini söyledi (Bölüm 11). Basit olsun diye derecesi 2 olan bir ikili ağaçla başladı. Her düğümden en fazla iki çocuk çıkabilir, üçüncü çocuk çıkamaz varsayımı vardır.

5.1 Sol ve Sağ Çocuk İndisi Tutmak

İlk fikir, temel bir dizi kullanmaktır. Dizinin her gözünde değer tutulur. Ama bir düğümün sol ve sağ çocuğunun nerede olduğunu bilmek için iki indis daha tutmak gerekir: sol çocuk (left child) indisi ve sağ çocuk (right child) indisi.

Hoca önce şöyle bir yerleşim çizdi: 1. gözün sol çocuğu 2. gözde, sağ çocuğu 3. gözdedir. 2'nin çocukları 4 ve 5, 3'ün çocukları 6 ve 7'dir. Ama her düğümün iki çocuğu olmak zorunda değildir. Örneğin 7. göz boş, 8. göz dolu olabilir. 4'ün sol çocuğu 8 iken sağ çocuğu yoksa, oraya -1 yazılabilir. Hoca, indislerin her zaman "iki katı ve iki katın bir fazlası" olmayacağını, zaman içinde boşluklar oluştukça yerleşimin farklılaşabileceğini söyledi.

Yeni eleman eklerken mümkün olduğunca köke yakın bir yere eklemek tercih edilir. Bunun için ilk boş yerin bilinmesi gerekir. Bir "boş" değişkeni (boş işaretçisi) sıradaki elemanın nereye girebileceğini tutar. Birden fazla boş yer varsa, kullanılabilir yerler ayrı bir **boş listesinde** tutulabilir.

5.2 Jane, Bob, Tom Örneği

Hoca bir aile ağacı örneği verdi. Jane büyük annedir. Onun iki çocuğu Bob ve Tom'dur. Bob'un iki çocuğu Alan ve Ellen'dır. Tom'un bir çocuğu Nancy'dir.

```
          Jane
         /    \
       Bob     Tom
      /   \    /
  Alan  Ellen Nancy
```

Dizide her gözde veri, sol evladın indisi ve sağ evladın indisi tutulur. Üçlü ağaç olsaydı birinci, ikinci ve üçüncü çocuk için üç indis, beşli ağaç olsaydı beş indis tutulurdu. Ayrıca ağaca erişebilmek için kökün indisi de bilinmelidir.

İndis	Veri	Sol	Sağ
0	Jane	1	2
1	Bob	3	4
2	Tom	5	-1
3	Alan	-1	-1
4	Ellen	-1	-1
5	Nancy	-1	-1

Kök 0, dizideki bir sonraki boş göz 6'dır. Tablo şöyle okunur: Jane'in sol evladı 1. gözdeki Bob, sağ evladı 2. gözdeki Tom'dur. Bob'un çocukları 3 ve 4'teki Alan ile Ellen'dır. Tom'un sol çocuğu 5'teki Nancy'dir, sağ evladı yoktur (-1).

Ekleme (Mary): Bir sonraki boş göz 6 olduğu için Mary 6. göze yazılır. Boş göz sayacı bir artırılır ve artık 7'yi gösterir. Mary, Nancy'nin sol çocuğu olacaksa, Nancy'nin sol indisi -1 yerine 6 yapılır. Artık Nancy'deyken "sol çocuğum 6. gözde" diye Mary'ye gidilebilir. Mary'nin çocuğu yoktur, yani bir yapraktır. Nancy ise artık bir ebeveyn düğüm olmuştur.

Silme (4. göz, Ellen): Silinecek göz boşaltılır. Bir sonraki boş yer artık 7 değil 4 olarak güncellenir. Ellen'a Bob'un sağ çocuğundan geliniyordu. Bob'un sağ indisindeki 4 silinip -1 yapılmalıdır. Hoca, 4 kalsa da 4. göz boş olduğu için bir çözüm olabileceğini, ama "sağ evladı yoktur" demenin, yani -1 yazmanın daha doğru olduğunu söyledi.

> **Ders sonrası not:** Silinen 4. göz doğrudan "bir sonraki boş göz" yapıldığında önceki boş göz bilgisi (7) kaybolur. Boş gözler bir zincirle tutuluyorsa, silinen gözün bağlantı alanına eski boş göz (7) yazılmalı, sonra boş göz 4 yapılmalıdır. Böylece $4 \to 7 \to 8 \to \ldots$ zinciri korunur.
>
> **Ek bilgi (derste anlatılmadı):** Boş gözleri birbirine bağlayarak tutan bu yapıya boş liste (free list) denir. Yeni eleman her zaman listenin başındaki gözü alır, silinen göz de listenin başına eklenir. Böylece hem ekleme hem silme sabit zamanda yapılır.

5.3 C ile Tanımlama

Bu yapı C'de şöyle tanımlanır: `data`, `left` ve `right` alanları olan bir `struct node`, bunlardan oluşan `tree` adında bir dizi (en fazla 100 eleman), kökün indisi ve bir sonraki boş gözün indisi.

```c
struct node {
    int data;
    int left;
    int right;
};
struct node tree[100];
int root, bos;
```

Hoca `bos` değişkenini neden Türkçe yazdığını bilmediğini, `empty` yazmanın daha doğru olacağını söyledi.

5.4 Konuma Göre Yerleştirmek

İkili ağaçta konumlar hiyerarşik olarak hesaplanabilir. 0 tabanlı dizide j indisli düğüm için:

- Sol çocuk $2j + 1$ indisindedir.
- Sağ çocuk $2j + 2$ indisindedir.
- Ebeveyn $\lfloor (j - 1)/2 \rfloor$ indisindedir. Bölme sonucu aşağı yuvarlanır.

Böylece yalnız dizi indisleriyle de ikili ağaç gösterilebilir.

5.5 Bellek İsrafı

Hoca bu gösterimin bedelini bir örnekle gösterdi. Dört elemanlı bir ağaç 1'den 7'ye kadar indisli bir diziye yerleştirilmiş, 4., 5. ve 7. gözler boş. Dizi olarak tutulduğu için 7 göz açılmalıdır, çünkü boş gözler de ileride dolabilir. Sekizinci eleman bir alt seviyeye eklenecekse, o

seviyenin bütün gözleri açılmalıdır ve 15 göze ihtiyaç duyulur. Bir seviye daha inildikçe 31, 63, 127 göz gerekir. Dizi olduğu için en kötü durumda en sona eleman eklenebilir. Aradaki gözler bomboş olsa bile yer ayrılır.

Hoca bu durumu "n tane eleman için $2^n - 1$ tane yer" diye özetledi. Üçlü ağaç için $3^n - 1$, dörtlü ağaç için $4^n - 1$ dedi.

> **Ders sonrası not:** "n eleman için $2^n - 1$ göz" ifadesi yalnız en kötü durumda, yani her düğümün tek çocuğu olduğu dejenere ağaçta geçerlidir. Genel ifade, yüksekliği h olan ikili ağaç için $2^{h+1} - 1$ gözdür (h = 0, 1, 2, 3 için 1, 3, 7, 15). Üçlü ağaç için doğru değer $3^{h+1} - 1$ değil $(3^{h+1} - 1)/2$'dir (h = 0, 1, 2, 3 için 1, 4, 13, 40).

Ağaç dolu ya da neredeyse doluysa dizide hiç boşluk ya da çok az boşluk olur. Hoca bu kadar 3-5 gözlük kaybın göz ardı edilebileceğini söyledi. Ama boşluklar çoksa ve ağaç tam dolu değilse çok fazla yer israf edilir.

Hoca bunu P, G, R, B, H, V, K düğümlerinden oluşan bir ağaçla gösterdi. Dizide 0'dan 14'e kadar 15 göz vardır. Bazı gözler (örneğin 5, 7, 8, 9) boştur.

En kötü durumda hiçbir sol çocuk yoktur. Hoca bunu ileride "PRVZ" diye andığı bir örnekle anlattı: P'nin yalnız sağ çocuğu R, R'nin yalnız sağ çocuğu V, V'nin yalnız sağ çocuğu Z vardır. 0 tabanlı yerleşimde bu düğümler 0, 2, 6 ve 14. indislere düşer. Hoca boş gözleri seviye seviye saydı: en alt seviyede 7, bir üstte 3, onun üstünde 1 göz boştur. Toplam 11 göz boştur. Yani 4 değer tutmak için 15'lik dizi açılmıştır ($2^4 - 1 = 16 - 1 = 15$) ve yalnız 4'ü doludur. Yükseklik arttıkça 2^n mertebesinde bellek gerekir.

Hocanın vardığı sonuç: Dizi gösterimi çok etkin bir gösterim değildir, yer kaybı çok fazladır ve bu sorunun çözülmesi gerekir.

6. Ağacın Bağlantılı Liste ile Gösterimi

Aradan sonra hoca çözümü verdi: dizi yerine bağlantılı liste (linked list) yapısından yararlanmak. Her düğümde iki işaretçi (pointer) bulunur. Biri sol alt çocuğun adresini, diğeri sağ alt çocuğun adresini tutar. Çocuk yoksa işaretçi `NULL`'dır.

Hoca slayttaki çizimde, sol çocuğu olup sağ çocuğu olmayan bir düğümü, iki çocuğu olan ara düğümleri ve iki işaretçisi de `NULL` olan dört yaprak düğümü gösterdi. Bağlantılı liste yapıları oldukça esnek olduğu için, n eleman için n tane `struct` kullanarak fazla yer harcamadan ağaç kurulabilir.

```
typedef struct node *tree;
struct node {
    char data;
    tree left;
    tree right;
};

tree createTree(char data) {
    tree t = (tree)malloc(sizeof(struct node));
    t->data = data;
    t->left = NULL;
    t->right = NULL;
    return t;
}
```

Önce `struct node` işaretçisi için `tree` adında bir tip tanımlanır (`typedef`). Bu sayede `struct` içinde sol ve sağ alt ağaçlar da `tree` tipiyle yazılabilir. Ağaç oluştururken yapılan iş şudur: bir düğüm kadar yer ayrılır (`malloc`), `t` işaretçisiyle bu düğüme erişilir, gelen veri yazılır, sol ve sağ alt ağaçlar boş (`NULL`) yapılır ve `t` çağırana döndürülür. Gerekirse bu düğümün soluna ya da sağına giderek yeni ağaçlar oluşturulur. Böylece eleman ekleme özyinelemeli (hiyerarşik) olarak yapılabilir.

> **Ders sonrası not:** Hoca yer ayırma için "size of tree kadar" dedi. `tree` burada bir

işaretçi tipidir. `sizeof(tree)` yalnız bir işaretçinin boyutunu verir, düğümün kendisine yetmez. Doğrusu `sizeof(struct node)`'dur.

Hoca, işaretçi kullanarak ikili ağacın bu şekilde oluşturulabileceğini, ya da herhangi bir ağacın çocuk ve kardeş düğümlerini gösterecek şekilde ifade edilebileceğini söyledi (Bölüm 11).

7. İkili Ağaçlar ve Özel Türleri

7.1 Özyinelemeli Tanım

İkili ağaçlar, genel ağaç veri yapısının bilgisayar bilimlerinde en çok kullanılan özel hâlidir. Hiçbir düğümün ikiden fazla çocuğu olamaz. Bu yüzden ikili ağaç özyinelemeli olarak şöyle tanımlanabilir:

- Boş küme bir ikili ağaçtır.
- Ya da bir kök düğüm ile buna bağlı, sol ve sağ alt ağaç adı verilen iki bağımsız ikili ağaçtan oluşan sonlu düğüm kümesi bir ikili ağaçtır.

Hoca slayttaki örneklerin her birinde kökü, sol alt ağacı (L) ve sağ alt ağacı (R) işaretleyerek bunların hepsinin birer ikili ağaç olduğunu gösterdi.

Bu tanımdan ikili ağaç için gereken işlem ailesi çıkar:

- Bir ikili ağaç yaratmak, ya da iki alt ağacı bir kökle birleştirerek ikili ağaç oluşturmak.
- Sol alt ağaca ve sağ alt ağaca erişmek.
- Bir düğümün verisine erişip kullanmak.

7.2 İkili Ağaç Türleri

İkili ağaçların farklı varyasyonları vardır. Bazılarında bütün düğümlerin iki çocuğu vardır, bazılarında eksikler vardır. Özel durumlara özel adlar verilir.

Full binary tree: Ağaçtaki her düğümün ya 0 ya da 2 çocuğu vardır. Yani tek çocuklu düğüm yoktur. Hocanın örneğinde kök 18'dir. 18'in iki çocuğu vardır. Diğer düğümlerin de ya iki çocuğu ya da hiç çocuğu yoktur. Ağaç yalnız yapraklardan ve tam iki çocuklu iç düğümlerden oluştuğu için full binary tree'dir.

Complete binary tree: Son seviye hariç bütün seviyeler tam doludur ve son seviye soldan başlayarak doldurulmuştur. Hocanın örneğinde kök tektir, kökün iki çocuğu vardır, bir alttaki dört düğüm de tam doludur. En alt seviyede normalde 8 çocuk beklenir, ama yalnız soldaki ilk üç göz doludur. Bu ağaçta tek çocuklu bir düğüm vardır, bu yüzden full değildir. Ama son seviye hariç bütün seviyeler tam dolu ve son seviye soldan dolu olduğu için complete'tir.

Hoca iki tanımı karşılaştırdı: Kökü 18 olan ağaç full'dür, ama bir seviyesi tam dolu olmadığı için complete değildir.

Perfect binary tree: Hem full hem complete'tir. Bütün iç düğümlerin iki çocuğu vardır ve bütün yapraklar aynı seviyededir. Kök, onun tam dolu iki çocuğu ve onların dörder çocuğuyla dolu bir ağaç örnektir. Hoca bunu "alabileceği bütün elemanları alarak dolmuş, mükemmel bir ikili ağaç" diye tanımladı.

Bu üçü ikili ağacın özel durumlarıdır. Bunların dışında bir tanım daha vardır:

Balanced binary tree: n düğümden oluşan bir ikili ağacın yüksekliği $\log n$ mertebesindeyse ağaç dengelidir (balanced).

Hoca örnek olarak "8 düğüm varken 3 seviyede kalıyorsa ağaç dengelidir" dedi.

> **Ders sonrası not:** 3 seviyeli bir ikili ağaç en fazla $2^3 - 1 = 7$ düğüm alır. 8 düğüm için en az 4 seviye gerekir. Burada hoca büyük olasılıkla yüksekliği kastetti: 8 düğümlü dengeli ağacın yüksekliği $\log_2 8 = 3$'tür.

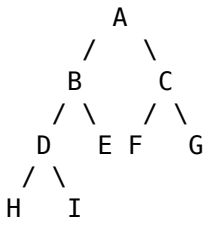
Dejenere (patolojik) ağaç (degenerate / pathological tree): Her iç düğümün tek bir çocuğu vardır. Hocanın PRVZ örneğinde her düğümün yalnız sağ çocuğu vardır. Bu yapı aslında bir bağlantılı liste olmuştur. Çocuklar bir sola, bir sağa giderek "yılan gibi" ilerlese de sonuç aynıdır: Her düğümün tek çocuğu olduğu için ağaç, iki bağlantılı değil, düz tek bağlantılı bir listeye dönüşür.

7.3 Bağlantılı Listeyle İki Örnek

Hoca iki ağacı bağlantılı listeyle çizdi.

Birinci örnekte kök A'dır. A'nın sol çocuğu B, B'nin sol çocuğu C, C'nin sol çocuğu D'dir ve D'nin çocuğu yoktur. Hepsi sola gitmiştir. Bu bir dejenere ikili ağaçtır.

İkinci örnek şudur:



Hoca bu ağacı türlere göre inceledi:

- **Full mü?** Herkesin ya 0 ya 2 çocuğu vardır. Evet, full binary tree'dir.
- **Complete mi?** Üstteki seviyeler tam doludur, en alt seviye soldan dolmuştur. Evet, complete binary tree'dir.
- **Perfect mi?** Alt seviye tam dolu değildir. Perfect değildir.
- **Dengeli mi?** Ağaçta 9 düğüm vardır. En alttaki yaprağa gitmek için 3 kenar geçilir, yani toplam 4 seviye vardır.

Hoca $\log_2 9$'u "4 nokta bir şey" diye hesapladı. Ardından ağacı önce dengeli olarak işaretledi, sonra "3, 4 mü oluyor, o zaman dengeli değil" deyip işaretini değiştirmeye çalıştı ve "logaritmayı alamadık" diyerek konuyu kapattı.

> **Ders sonrası not:** $\log_2 9 \approx 3{,}17$'dir, "4 nokta bir şey" değildir ($2^3 = 8 < 9 < 16 = 2^4$). Ağacın yüksekliği 3'tür ve bu $\log_2 9$ ile aynı mertebededir. Dolayısıyla ağaç dengelidir. Zaten 9 düğümlü bir ikili ağacın alabileceği en küçük yükseklik $\lfloor \log_2 9 \rfloor = 3$'tür, yani bu ağaç olabilecek en kısa yüksekliğe sahiptir.

8. İkili Ağaca Eleman Ekleme ve Silme

8.1 Düğüm Oluşturma ve Kullanma

İkili ağaç oluşturmak için Bölüm 6'daki kod kullanılır. Örneklerde A, B, C gibi harfler kullanıldığı için veri karakter olarak tutulur. Sol ve sağ alt ağaçlar `struct node` işaretçileriyle gösterilir.

Ağaçtaki bir düğüm "kullanılırken" yapılacak iş size kalmıştır. Örneğin `printf` ile verisi ekrana yazdırılabilir ya da veri döndürülebilir. Hoca bu işleme **ziyaret (visit)** dedi: düğümün verisi üzerinde istenen işlem yapılır.

8.2 Ekleme

Eleman eklerken bir hiyerarşiye ihtiyaç vardır. Hocanın önerdiği en basit hiyerarşi küçüklük-büyüklük ilişkisidir: Karakter ya da tamsayı verisinde küçükler sol çocuk, büyükler sağ çocuk olarak eklenir. Tersini kurgulamak da mümkündür, bu bize kalmıştır.

```
tree insert(tree node, char data) {
    if (node == NULL)
        return createTree(data);
    if (data < node->data)
        node->left = insert(node->left, data);
    else if (data > node->data)
        node->right = insert(node->right, data);
    return node;
}
```

Hoca örneği adım adım yürüttü. Ağaç 6 değeriyle oluşturulur. Sol ve sağ işaretçileri `NULL`'dır.

1. **4 eklenir:** Düğüm boş değildir. 4, 6'dan küçük mü? Evet. `insert(node->left, 4)` çağrılır. Sol `NULL` olduğu için orada `createTree(4)` ile yeni düğüm oluşturulur ve 4 yazılır.
2. **5 eklenir:** Kök 6'dan başlanır. 5, 6'dan küçüktür, sola gidilir. 4'lü düğümde: 5, 4'ten küçük mü? Hayır. Büyük mü? Evet. Sağa eklenir. Sağ `NULL` olduğu için 5, 4'ün sağ çocuğu olarak oluşturulur.
3. **8 eklenir:** Kök 6'dan başlanır. 8, 6'dan küçük değil, büyüktür. 6'nın sağı `NULL` olduğu için 8 oraya eklenir.
4. **4 tekrar eklenir:** 4, 6'dan küçüktür, sola gidilir. 4'lü düğüm `NULL` değildir. 4 ne küçüktür ne büyüktür. Hiçbir şey yapılmaz.

Sonuç ağacı:

```
    6
   / \
  4   8
   \
    5
```

Hoca, ikinci koşulda `else` yazmanın da yeteceğini söyledi. Ama eşitlik durumunda hiçbir şey yapılmayacağı için koşul `else if` olarak yazılmıştır. Tekrarlanan değerlerin de eklenmesi gerekiyorsa başka bir yol izlenmelidir.

8.3 Silme

Hoca silmeyi, az önce oluşan ağaca birkaç düğüm daha ekleyerek anlattı.

- **Yaprak silmek:** Yaprak doğrudan silinebilir. Ebeveynindeki ilgili işaretçi (örneğin sağ işaretçi) `NULL` yapılır ve düğüm `free` edilir. Başka bir şey yapmaya gerek yoktur.
- **Tek çocuklu ara düğüm silmek:** Ara düğüm silinirken dikkatli olmak, çocukların durumuna bakmak gerekir. Düğümün tek çocuğu varsa iki yol vardır: Ya çocuk silinen düğümün yerine taşınır, ya da çocuk, silinen düğümün ebeveynine bağlanır ve düğüm silinir. İkisi de olur.
- **İki çocuklu düğüm silmek:** Düğümün iki çocuğu ve onların da alt soyu varsa, silinen değerin yerine hangi değerin taşınacağına karar vermek gerekir. Hiyerarşinin bozulmaması için ya sol alt ağaçtaki en büyük değer ya da sağ alt ağaçtaki en küçük değer oraya taşınmalıdır.

Hoca bu üç durumun ikili arama ağacı konusunda (Bölüm 12.4) ayrıntılı ele alınacağını söyledi.

9. Ağaçta Gezinti

Elimizde eleman ekleyip silebildiğimiz bir ikili ağaç var. Hocanın sorusu şuydu: Bu ağaçtaki elemanları nasıl gezebilir, onlara erişip nasıl kullanabiliriz? Gezintiye her zaman kökten başlanır, çünkü başka hiçbir yerden erişilemez.

9.1 Derinlik Öncelikli ve Genişlik Öncelikli

Temelde iki gezinti (traversal) yaklaşımı vardır:

- **Derinlik öncelikli gezinti (depth first traversal):** Önce aşağı doğru inilebildiği kadar inilir, yapraklara erişilir ve onlar öncelikli olarak kullanılır.
- **Genişlik öncelikli gezinti (breadth first traversal):** Ağacın seviyeleri gözetilir. Her seviye kendi içinde, seviye sırasıyla gezilir.

9.2 Altı Olası Sıra ve Üç Temel Gezinti

Hoca derinlik öncelikli gezintiyi en temel yapı üzerinden kurdu: bir ebeveyn (P), onun sol (L) ve sağ (R) çocuğu. Bir düğümde yapılabilecek üç iş vardır: ziyaret (V), sola gitmek (L), sağa gitmek (R). Bunların sıralanışına göre altı gezinti çıkar:

- **V L R:** Önce düğüm kullanılır, sonra sola, sonra sağa gidilir.
- **V R L:** Bunun aynası (mirror). Önce sola gitmek yerine önce sağa gidilir.

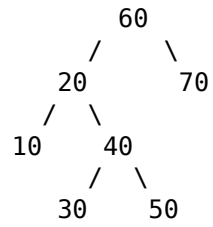
- **L V R:** Önce sol alt ağaca gidilebildiği kadar gidilir, sonra düğüm kullanılır, sonra sağa gidilir.
- **R V L:** Bunun aynası.
- **L R V:** Önce sola, sonra sağa gidilir, düğüm en son kullanılır.
- **R L V:** Bunun aynası.

Ayna sıralar, aynı gezintinin sağdan sola yapılmış hâlidir. Hocanın ifadesiyle, ağaca kâğıdın arkasından bakınca sağ taraf sol taraf olur. Bu yüzden üçü silinir ve geriye üç temel gezinti kalır. Adları, ziyaretin (visit) sıradaki yerinden gelir:

- **V L R → preorder (kök önde):** Önce kullan, sonra gezintiye devam et.
- **L V R → inorder (kök ortada):** Ziyaret ortadadır.
- **L R V → postorder (kök sonda):** Sola ve sağa dallanıldıktan sonra en son ziyaret edilir.

9.3 Preorder Gezinti

Hocanın gezintiler için kullandığı örnek ağaç şudur:



Temel fikir: Önce kök ziyaret edilir. Sonra sol alt ağaç kök önde olarak, sonra sağ alt ağaç kök önde olarak özyinelemeli gezilir. Alt ağaçlarda da her seferinde önce kendi kökleri ziyaret edilir.

Hoca adımları tek tek yürüttü:

1. Kökteyiz. 60 ziyaret edilir ve kullanılır.
2. Sola dallanılır. Artık 20'nin köklü alt ağacındayız. 20 ziyaret edilir.
3. Hâlâ sola gidilebiliyor. 10'un alt ağacına gidilir. 10 ziyaret edilir.
4. 10'un solu `NULL`'dır, geri dönülür. Sağı da `NULL`'dır. 10'un alt ağacı bitti, çağırana dönülür.
5. 20'nin ziyareti ve solu bitti. Sağı çağrılır: 40 ziyaret edilir.
6. 40'ın soluna gidilir: 30 ziyaret edilir. Solu ve sağı yoktur, geri dönülür.
7. 40'ın sağına gidilir: 50 ziyaret edilir. Solu ve sağı yoktur, geri dönülür.
8. 40'ın VLR'si, sonra 20'nin VLR'si biter. Köke dönülür.
9. Kökün ziyareti ve solu bitti. Sağına gidilir: 70 ziyaret edilir. Solu ve sağı yoktur.
10. Kökün VLR'si tamamlanır.

Sonuç: **60, 20, 10, 40, 30, 50, 70.**

```c
void preTraverse(tree t) {
    if (t == NULL) return;
    printf("%c", t->data);   /* visit */
    preTraverse(t->left);
    preTraverse(t->right);
}
```

`t` `NULL` ise geri dönülür. Bu, özyinelemenin durma şartıdır. Hocanın "solu yok, sağı yok" dediği temel durum budur. Sonra düğüm ziyaret edilir, ardından fonksiyon kendini önce sol, sonra sağ alt ağaç için çağırır.

9.4 Inorder Gezinti

Sıra bu kez sol, ziyaret, sağ (L V R) şeklindedir. Hoca adımları aynı ağaçta yürüttü:

1. Kökten başlanır. Kökün soluna inorder çağrılır: 20'deyiz.
2. 20'nin solu vardır: 10'a gidilir. 10'un solu `NULL`'dır, geri dönülür. 10 ziyaret edilir. Sağı `NULL`'dır. 10'un LVR'si biter.
3. 20'ye dönülür. Sol alt ağaç bitti. 20 ziyaret edilir.

4. 20'nin sağına gidilir: 40'tayız. 40'ın soluna gidilir: 30'un solu yoktur, 30 ziyaret edilir, sağı yoktur.
5. 40'a dönülür. 40 ziyaret edilir. 40'ın sağına gidilir: 50'nin solu yoktur, 50 ziyaret edilir, sağı yoktur.
6. 40'ın, sonra 20'nin LVR'si biter. Köke dönülür. Kökün solu bitti. 60 ziyaret edilir.
7. Kökün sağına gidilir: 70'in solu yoktur, 70 ziyaret edilir, sağı yoktur.
8. Kökün LVR'si tamamlanır.

Sonuç: **10, 20, 30, 40, 50, 60, 70.**

```
void inTraverse(tree t) {
    if (t == NULL) return;
    inTraverse(t->left);
    printf("%c", t->data);   /* visit */
    inTraverse(t->right);
}
```

9.5 Postorder Gezinti

Hoca postorder kodunu öğrencilere bırakarak başladı, sonra gezintiyi yine adım adım yaptı. Sıra sol, sağ, ziyaret (L R V) şeklindedir.

1. Kökten 20'ye, oradan 10'a gidilir. 10'un solu ve sağı boştur. 10 ziyaret edilir.
2. 20'ye dönülür. Solu bitti, sağına (40) gidilir. 40'ın soluna gidilir: 30'un solu ve sağı yoktur. 30 ziyaret edilir.
3. 40'a dönülür, sağına gidilir: 50'nin solu ve sağı yoktur. 50 ziyaret edilir.
4. 40'ın sağ alt ağacı bitti. 40 ziyaret edilir.
5. 20'nin sağ alt ağacı da bitti. 20 ziyaret edilir.
6. Kökün sol tarafı bitti. Sağına gidilir: 70'in solu ve sağı `NULL`'dır. 70 ziyaret edilir.
7. Kökün solu ve sağı bitti. 60 ziyaret edilir.

Sonuç: **10, 30, 50, 40, 20, 70, 60.** Hoca sonucu iki kez okuyarak kontrol etti.

```
void postTraverse(tree t) {
    if (t == NULL) return;
    postTraverse(t->left);
    postTraverse(t->right);
    printf("%c", t->data);   /* visit */
}
```

Hoca "yazdır" dediği yerlerde, gerçek bir algoritmada değerlerin bu sırayla kullanılacağını hatırlattı. Ayna gezintiler için yalnız çağrı sırası değişir. Örneğin R L V için önce `t->right`, sonra `t->left` çağrılır, sonra ziyaret edilir.

9.6 Yığınla Özyinelemesiz Inorder Gezinti

Hoca özyinelemeli yapı kullanmak istemeyenler için bir alternatif gösterdi. Nerede kalındığını ters sırada işlemek için tutmak gerekir. Bunun için geçen hafta görülen yığın (stack) yapısı, yani son giren ilk çıkar (last in, first out) kullanılabilir.

Algoritmanın adımları şunlardır:

1. `current`, dolaşılan düğümün işaretçisidir. Kökten (`root`) başlanır.
2. `current` `NULL` olmadığı sürece düğümün adresi yığına itilir (`push`) ve `current`'ın soluna gidilir. Yani gidilebildiği kadar sola gidilir.
3. `current` `NULL` olduğunda ve yığın boş değilse yığından bir düğüm çekilir (`pop`) ve yazdırılır. Bu, ziyarettir.
4. `current` çekilen düğümün sağ çocuğu olarak güncellenir ve 2. adıma dönülür.
5. `current` `NULL` ve yığın boş olduğunda bütün elemanlar sol, ziyaret, sağ sırasıyla gezilmiş olur.

```
void inTraverseStack(tree root) {
    tree current = root, x;
    while (current != NULL || !isEmpty()) {
```

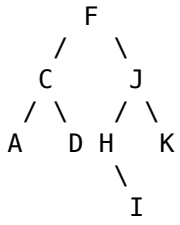
```
        while (current != NULL) {
            push(current);
            current = current->left;
        }
        x = pop();
        printf("%c", x->data);   /* visit */
        current = x->right;
    }
}
```

Hocaya göre burada yapılan şey, özyinelemeli çözümde fonksiyon çağrı yığınının (function call stack) yaptığı işi elle yapmaktır. Kök önce yığına girer, ama soldakiler bitirilip geri dönüldükten sonra en son ziyaret edilir.

9.7 Genişlik Öncelikli Gezinti

Derinlik öncelikli değil de genişlik öncelikli, yani seviye öncelikli gezinti için kuyruk (queue), yani ilk giren ilk çıkar (first in, first out) yapısı kullanılır. Ağaç seviye sırasında, soldan sağa dolaşılır.

Hocanın örnek ağacı şudur:



Adımlar:

1. Kök F kuyruğa eklenir.
2. F kuyruktan çıkarılır ve kullanılır. Çocukları soldan sağa kuyruğa eklenir: C, J.
3. C çıkarılır ve kullanılır. Çocukları A ve D eklenir.
4. J çıkarılır (dequeue) ve kullanılır. Çocukları H ve K eklenir (enqueue).
5. A ve D çıkarılır ve kullanılır. Çocukları yoktur.
6. H çıkarılır ve kullanılır. Çocuğu I eklenir.
7. K çıkarılır, sonra I çıkarılır ve kullanılır. Kuyruk boşalır.

Genel kural: Önce kök kuyruğa konur. Kuyrukta eleman olduğu sürece bir eleman çıkarılır, kullanılır ve onun çocukları sol-sağ sırasıyla kuyruğa eklenir. Her seviyedeki çocuklar soldan sağa eklendiği için önce F, sonra C ve J, sonra A, D, H, K ve en son alt seviyedeki tek çocuk I işlenir.

Sonuç: **F, C, J, A, D, H, K, I.**

> **Ders sonrası not:** Hoca sonucu sesli okurken “F, C, C, A, D, H, K, I” dedi. İkinci C yerine J olmalıdır: F, C, J, A, D, H, K, I.
>
> **Ders sonrası not:** Hoca genişlik öncelikli gezintiyi “her seviyede bir inorder gezinti” diye tanımladı. Bu gezintinin inorder (L V R) ile ilgisi yoktur. Doğru ad seviye sırası gezintisidir (level order traversal): her seviye soldan sağa gezilir.

10. Ağaç Kopyalama ve İki Ağacın Eşitliği

10.1 Ağaç Kopyalama

Elimizde yalnız kökünün işaretçisi olan orijinal bir ağaç vardır. Hoca kopyalamayı şöyle anlattı: Orijinal ağaçtan bir `tmp` düğümü yaratılır. `tmp`’nin solu, orijinalin sol alt ağacı için kopyalama fonksiyonu yeniden çağrılarak elde edilir. Sağı da aynı şekilde sağ alt ağaç kopyalanarak elde edilir. Veri de orijinal düğümden alınıp yeni düğüme konur.

Yani sıra sol, sağ, ziyaret şeklindedir. Hoca "Kopyalamada ziyareti neden en sona koymalıyım?" diye sordu ve cevabını verdi: Önce veri yapısı, yani düğümler oluşturulmalıdır. Ardından veri oraya kopyalanmalıdır. Bu yüzden ağaç kopyalama postorder gezintiyle rahatlıkla yapılır.

Hocanın anlatımına göre kod şöyle kurulabilir (ara adım eklendi: `NULL` kontrolü ve yer ayırma):

```
tree copyTree(tree original) {
    tree tmp;
    if (original == NULL)
        return NULL;
    tmp = (tree)malloc(sizeof(struct node));
    tmp->left = copyTree(original->left);
    tmp->right = copyTree(original->right);
    tmp->data = original->data;   /* visit */
    return tmp;
}
```

> **Ek bilgi (derste anlatılmadı):** Kopyalama preorder sırayla da yazılabilir: önce düğüm oluşturulur ve verisi yazılır, sonra sol ve sağ alt ağaçlar kopyalanır. Her iki sırada da yeni düğüm, kendi alt ağaçlarına bağlanmadan önce oluşturulur. Postorder ise ağacı silerken (`free`) zorunludur, çünkü bir düğüm, çocukları serbest bırakılmadan silinirse çocuklara erişim kaybolur.

10.2 İki Ağacın Eşitliği

Hoca iki ağaç (`first` ve `second`) verip sınıfa şu soruyu sordu: Bu iki ağacın eşit olup olmadığını test etmek için preorder, inorder ve postorder gezintilerinden hangisi daha uygundur?

Soru: Inorder mı?

Cevap: Hoca inorder'ı birlikte düşünmeyi önerdi: Inorder'da önce sola, sola, sola gidilir, sonra ziyaret edilir. Eşitlik kontrol edilirken ise önce değer karşılaştırması yapılmalıdır. O zaman ziyaret, sol, sağ sırası daha doğru olur. Sınıftan "preorder daha doğru olur" cevabı geldi. Hoca bunu onayladı: Eğer kökte bir eşitsizlik varsa alt ağaçlara inmeden doğrudan çıkılır.

```
int isEqualTree(tree first, tree second) {
    return ((!first && !second) ||
            (first && second &&
             first->data == second->data &&
             isEqualTree(first->left, second->left) &&
             isEqualTree(first->right, second->right)));
}
```

Fonksiyon doğrudan doğru ya da yanlış döndürür. İkinci şart şunları ister: İki ağaç da `NULL` olmamalı, `first`'ün verisi `second`'ın verisine eşit olmalı (bu, ziyarettir), sol alt ağaçlar birbirine eşit olmalı ve sağ alt ağaçlar birbirine eşit olmalı. Ziyaret, sol, sağ sırasıyla kontrol yapıldığı için preorder bir gezintidir.

> **Ders sonrası not:** Hoca ilk şart (`!first && !second`) için "bunlardan herhangi biri `NULL` ise eşitlik olamaz" dedi. Bu şart, iki ağacın **ikisinin birden** boş olduğu durumdur ve doğru (1) döndürür: iki boş ağaç eşittir. Yalnız biri boşsa ilk şart yanlış olur, ikinci parantezdeki `first && second` da yanlış olur ve fonksiyon yanlış (0) döndürür.

Hoca bu iki örneğin preorder ve postorder için olduğunu söyledi. Inorder için de gezinti örneğindeki ağaca geri döndü: O ağaç aslında bir ikili arama ağacı (binary search tree) olarak kurulmuştur. Her elemanın küçükleri solunda, büyükleri sağında yer alır. Böyle bir ağaçta inorder gezinti yapılınca sıralı dizi (sorted array) kolayca elde edilir: 10, 20, 30, 40, 50, 60, 70.

11. Çok Çocuklu Ağaçların İkili Ağaç ile Gösterimi

Hoca ardından çok çocuklu ağaçlara (n-ary tree, Türkçede çoklu ya da çok evlatlı ağaç) geçti. Slayttaki ağaçta düğümlerden 3, 2, 1 çocuk çıkıyordu, en fazla çocuk sayısı 6 idi. Hoca bunu "6 seviyeli bir ağaç" diye okudu.

> **Ders sonrası not:** Hoca burada da “6 seviyeli” ve sonraki örnekte “ağacın seviyesi dört” dedi. Kastedilen derecedir: bir düğümün en fazla 6 (sonraki örnekte 4) çocuğu vardır.

Hocaya göre bu yapı çok kolay bir şekilde ikili ağaç olarak ifade edilebilir. Her düğüm için yalnız iki bilgi yeterlidir:

- **İlk çocuk (first child):** Düğümün ilk çocuğu kimdir?
- **İlk kardeş (next sibling):** Düğümün sağındaki ilk kardeşi kimdir?

Birinci örnekte:

- A’nın ilk çocuğu B’dir. B’nin kardeşleri sırasıyla C, D, E, F, G’dir.
- B’nin ilk çocuğu H’dir, ilk kardeşi C’dir. H’nin kardeşi I’dır.
- C’nin ilk çocuğu yoktur, ilk kardeşi D’dir. D’nin ilk çocuğu yoktur, ilk kardeşi E’dir.
- E’nin ilk çocuğu K’dır, K’nın kardeşi L’dir. K’nın çocuğu P, L’nin çocuğu Q’dur.
- G’nin çocuğu M’dir.

Yapı birebir aynıdır, ama yalnız ilk çocuğu ve ilk kardeşi bilmek yeterlidir. İkili ağaçta `data`, `left` ve `right` kullanılıyordu. Burada da aynı düğüm yapısıyla, biri veri ikisi işaretçi olmak üzere `data`, `child` ve `sibling` alanlı bir düğüm yeterlidir. Demek ki ağacın derecesi ne olursa olsun, ağaç ikili ağaç olarak ifade edilebilir.

Hoca ikinci örneği, çizmesi zor geldiği için elle çizdiği dörtlü bir ağaçla verdi. Düğümlerden 4, 2, 3 ve 1 çocuk çıkıyordu. Çocuk işaretçisi aşağı, kardeş işaretçisi sağa doğru çizildi:

- Kök V0’dır. V0’ın ilk çocuğu V1’dir. V1’in kardeşleri V2, V3, V4’tür.
- V1’in ilk çocuğu V5’tir, V5’in kardeşi V6’dır.
- V3’ün ilk çocuğu V7’dir, V7’nin kardeşleri V8 ve V9’dur.
- V7’nin ilk çocuğu V10’dur. Geri kalan işaretçiler `NULL`’dır.

```
V0
|
V1 -> V2 -> V3 -> V4
|           |
V5 -> V6    V7 -> V8 -> V9
            |
            V10
```

Bu düz metin çizimde aşağı ok yerine `|` ilk çocuğu, `->` bir sonraki kardeşi gösterir.

Hocanın vardığı sonuç: Şimdiye kadar ikili ağaç için söylenen oluşturma, gezinti, eleman ekleme ve silme işlemlerinin hepsi bütün ağaçlara genelleştirilebilir. Bir ağaç ikili ağaç olarak gerçekleştirildiği anda, aynı fonksiyonlarla üzerinde gezinti ve işlem yapılabilir.

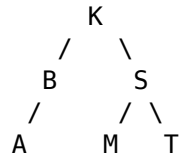
12. İkili Arama Ağacı

12.1 Tanım

İkili ağacın sık kullanılan örneklerinden biri ikili arama ağacıdır (binary search tree, BST). Her düğüm şu özellikleri taşır:

- Düğümün değeri, sol alt ağacındaki bütün düğümlerin değerlerinden büyüktür.
- Düğümün değeri, sağ alt ağacındaki bütün düğümlerin değerlerinden küçüktür.
- Sol alt ağaç ve sağ alt ağaç da birer ikili arama ağacıdır. Yani bu özellikler alt ağaçlarda da aynen geçerlidir.

Hoca önce alfabetik bir örnek verdi. Kökte K harfi vardır. Solunda B (K’dan önce gelir), sağında S (K’dan sonra gelir) vardır. A, B’den küçüktür ve B’nin solundadır. M, S’den küçüktür ve S’nin solundadır. T, S’den büyüktür ve S’nin sağındadır.



Sol alt ağaçtaki B ve A, K'dan küçüktür. Sağ alt ağaçtaki S, M ve T, K'dan büyüktür. Sağa dallandıkça büyük, sola dallandıkça küçük elemanlara gidilir. Ama her alt ağaç, yukarıdaki atalarıyla olan büyüklük-küçüklük ilişkisini de korur.

Hoca aynı şeyi sayılarla da gösterdi. Kökü 21 olan ağaçta sol alt ağaçta 21'den büyük, sağ alt ağaçta 21'den küçük hiçbir sayı yoktur. 34, 3 ve 8 düğümleri için de aynı şey geçerlidir. Bütün alt ağaçlar birer ikili arama ağacı olduğu için iki örnek de ikili arama ağacıdır.

12.2 Hangisi İkili Arama Ağacıdır?

Hoca slaytta üç ağacı tek tek kontrol etti:

- **Kökü 20 olan ağaç (15, 12, 10 değerleri):** 15, 12 ve 10, 20'den küçüktür. Ama 10'un bulunduğu yerde 15'ten büyük olması gerekirdi. 10 bu kuralı bozar. Hoca ayrıca ikinci bir bozulma daha gösterdi. Bu ağaç ikili arama ağacı değildir. Hoca bunu "A şıkkı" diye andı.
- **Kökü 30 olan ağaç:** 40 sağdadır ve daha büyüktür. 5 soldadır ve daha küçüktür. 2, 5'in solundadır ve 5'ten de küçüktür. Bu ağaç dengesizdir ama ikili arama ağacıdır. Hoca, şu an dengeden konuşmadığımızı söyledi.
- **Kökü 60 olan ağaç:** Sol tarafı yoktur, 60'tan küçük eleman yoktur. 60'ın sağında 70, 70'in sağında 80 vardır. 70'in solunda 65 vardır. 65, 60'tan büyük ve 70'ten küçük olduğu için doğru yerdedir. Bu da bir ikili arama ağacıdır.

Gözetilmesi gereken kural hep aynıdır: Kökteki değerin sol tarafındakilerin hepsi daha küçük, sağ alt ağaçtakilerin hepsi daha büyük olmak zorundadır.

12.3 Arama

Eleman ararken önce kökteki verinin aranan değerden büyük mü küçük mü olduğuna bakılır.

- Kökteki veri büyükse, aranan eleman daha küçüktür. Küçük elemanlar ikili arama ağacında sol taraftadır. Fonksiyon kendini sol alt ağaç için çağırır ve arama orada devam eder.
- Kökteki veri büyük değilse küçük mü diye bakılır. Küçükse, aranan veri daha büyüktür ve sağ alt ağaçtadır. Arama sağ alt ağaçta devam eder.
- Dallanma, bir eşitlik bulunana ya da `NULL` görülene kadar sürer. Bu yüzden kontrol yapmadan doğrudan `root` döndürülebilir: Değer bulunduysa düğüm, bulunmadıysa `NULL` döner.

```
tree search(tree root, int data) {
    if (root == NULL || root->data == data)
        return root;
    if (data < root->data)
        return search(root->left, data);
    return search(root->right, data);
}
```

Bu fonksiyon yalnız "var ya da yok" bilgisi verir. Hoca, değerin konumu, indisi ya da seviyesi gerekiyorsa kodun buna göre geliştirilebileceğini söyledi.

Hoca kökü 44 olan bir ağaçta iki arama yaptı.

25'i aramak:

1. Kök 44'tür. 25, 44'ten küçüktür. Sol alt ağaca dallanılır: 17.
2. 25, 17'den büyüktür. Sağ alt ağaca dallanılır: 32.
3. 32, 25'ten büyüktür. Sol alt ağaca dallanılır: 28.
4. 28, 25'ten büyüktür. Sol alt ağaca dallanılır: `NULL`.
5. `NULL` döner. 25 bu ağaçta yoktur.

Yol: 44 → 17 → 32 → 28 → `NULL`.

Hoca bu adımda bir an "bu şart yanlış" deyip hemen "pardon, bu şart doğru" diye düzeltti: Kökteki veri (44) aranan veriden (25) büyük olduğu için sola gitme şartı doğrudur.

76'yı aramak:

1. 44’ten başlanır. 76 daha büyüktür. Varsa sağ alt ağaçtadır: 88.
2. 88, 76’dan büyüktür. Sol alt ağaca gidilir: 65.
3. 65, 76’dan küçüktür. Sağına bakılır: 82.
4. 82, 76’dan büyüktür. Soluna bakılır: 76.
5. Değer bulunur ve döndürülür.

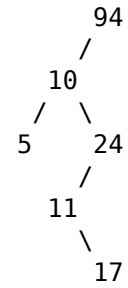
Yol: 44 → 88 → 65 → 82 → 76.

12.4 Ekleme

Ekleme Bölüm 8.2’deki kodla aynıdır. Düğüm `NULL` ise `malloc` ile yeni düğüm oluşturulur, veri konur, sol ve sağı `NULL` yapılır. Veri daha küçükse sola, daha büyükse sağa eklenir. En sonunda düğüm çağırana döndürülür.

12.5 Silme

Hoca silmenin üç durumunu kökü 94 olan bir ağaç üzerinde anlattı. Altyazıdaki adımlardan çıkan ağaç yapısı şudur:



Durum 1, yaprak silmek (5): Önce 5 bulunur: 94’ten 5 küçüktür, sola gidilir. 10’dan küçüktür, sola gidilir. 5 bulunur. Solu ve sağı yoktur, yani bir yapraktır. Doğrudan silinir ve `free` edilir. Bu silmeden hiçbir düğüm etkilenmez. Bir elemanın iki çocuğu da `NULL` ise o bir yapraktır ve silmek en kolay durumdur.

Durum 2, tek çocuklu düğüm silmek (24): Önce 24 bulunur: 94’ten küçük, sola gidilir. 10’dan büyük, sağa gidilir. 24’ün sağ çocuğu yoktur, sol çocuğu vardır. Sol çocuğun alt soyu bile vardır, ama sonuçta tek çocuk vardır. 24’e 10’dan gelinmiştir. 24’ün çocuğu yukarıya, 10’a bağlanır ve 24 `free` edilir. Hoca bunu bir benzetmeyle anlattı: “Baba öldü, çocuğun velayetini dedeye vermiş olduk.” Ara düğüm kaldırılmış, alt soyu bir üst soya bağlanmıştır.

Aynı durumun sağ çocuklu karşılığı 11’dir. 11 bulunur (sola, sağa, sola gidilerek). 11’in sağ çocuğu vardır, sol çocuğu yoktur. Çocuk dedeye bağlanır, 11 silinir.

Durum 3, iki çocuklu düğüm silmek (10): Önce 10 bulunur: 94’ten küçük, sola gidilir. 10’un sol çocuğu da sağ çocuğu da `NULL` değildir. İki çocuğu vardır. Bunu silmek biraz daha zordur.

Hoca 10’un sol alt soyunu yeşille, sağ alt soyunu morla, “alt soyu kocaman” olacak şekilde çizdi. Silinen değerin yerine ne yazılmalıdır? Hiyerarşinin bozulmaması için iki seçenek vardır:

- **Sağ alt ağaçtaki (mor) en küçük eleman** buraya çekilir. Böylece sağ taraftaki herkesin bu değerden büyük kalması garanti olur.
- **Sol alt ağaçtaki (yeşil) en büyük eleman** buraya çekilir. Böylece bu değer, sol tarafın geri kalanının hepsinden büyük, sağ tarafın hepsinden küçük olmaya devam eder.

İkisi de doğrudur. Hangisi seçilirse o yapılır, ama ikisinden biri yapılmalıdır. Seçilen değer yukarı kopyalanır ve o değerin bulunduğu düğüm silinir. Bunun için `findMin` (sağdaki en küçüğü bulmak) ya da `findMax` (soldaki en büyüğü bulmak) fonksiyonları gerekir.

Sağ alt ağacın en küçüğüyle: 10’un sağ alt ağacına gidilir, yani 24 kökündeyiz. En küçük eleman, `NULL` görülene kadar gidilebildiği kadar sola gidilerek bulunur: sol, sol, sol. 24’ün solu 11’dir. 11’in solu yoktur. O hâlde en küçük eleman 11’dir. 11, 10’un yerine kopyalanır. Artık 10 değeri yoktur. Sonra eski 11 düğümü silinmelidir. Bu silme de özyinelemelidir: 11’in iki çocuğu, tek çocuğu ya da hiç çocuğu olabilir. Burada 11’in tek (sağ) çocuğu 17 yukarı bağlanır ve 11 silinir.

Sol alt ağacın en büyüğüyle: 10'un sol alt ağacına gidilir ve `NULL` görülene kadar sağa gidilir. Burada tek bir düğüm vardır: 5. 5 bir yapraktır. 5, 10'un yerine kopyalanır ve yaprak olan 5 silinir. Bu durumda 11 yerinde durmaya devam eder.

Silmeden sonra ağaç iki seçenekte şöyle olur:

Seçenek	10'un yerine gelen	Preorder	Inorder
Sağ alt ağacın en küçüğü	11	94, 11, 5, 24, 17	5, 11, 17, 24, 94
Sol alt ağacın en büyüğü	5	94, 5, 24, 11, 17	5, 11, 17, 24, 94

Her iki durumda da inorder sıra sıralı kalır, yani ağaç ikili arama ağacı olmaya devam eder.

findMin ve findMax: `findMin`, `NULL` olmadığı ve sola gidilebildiği sürece sola gider. `findMax`, `NULL` olmadığı sürece sağa gider.

```
tree findMin(tree t) {
    if (t != NULL)
        while (t->left != NULL)
            t = t->left;
    return t;
}

tree findMax(tree t) {
    if (t != NULL)
        while (t->right != NULL)
            t = t->right;
    return t;
}
```

Silme kodu: Hoca kodu şöyle anlattı:

1. Ağaç yoksa `NULL` döndürülür.
2. Silinecek veri kökteki veriden küçükse sola gidilip silme yapılır. Büyükse sağa gidilip silme yapılır. Yani eleman ya soldadır ya sağdadır, aranır ve bulunur. Bulunamazsa zaten yoktur, `NULL` döner.
3. Eleman bulunduysa ve iki çocuğu birden varsa, sağdaki en küçük değer bulunur, düğüme kopyalanır ve o değer sağ alt ağaçtan silinir.
4. İki çocuğu birden yoksa ya tek çocuğu vardır ya hiç çocuğu yoktur. Var olan çocuk yukarı alınır. Hiç çocuğu yoksa `NULL` kopyalanmış olur, düğüm de `NULL` olur.
5. `T` döndürülür.

```
tree delete(tree T, char data) {
    tree tmp;
    if (T == NULL)
        return NULL;
    if (data < T->data)
        T->left = delete(T->left, data);
    else if (data > T->data)
        T->right = delete(T->right, data);
    else if (T->left != NULL && T->right != NULL) {
        T->data = findMin(T->right)->data;
        T->right = delete(T->right, T->data);
    }
    else {
        tmp = T;
        T = (T->left != NULL) ? T->left : T->right;
        free(tmp);
    }
    return T;
}
```

Ders sonrası not: Hoca 4. adımı anlatırken "soldaki `NULL` ise left'i al, değilse right'ı al" dedi. Bu ters söylenmiştir. Sol `NULL` ise sağ çocuk alınmalıdır. Doğrusu: Sol `NULL` değilse sol çocuk, değilse sağ çocuk alınır (`T = (T->left != NULL) ? T->left : T->right`).

Ders sonrası not: Hocanın anlattığı kodda tek çocuklu ya da yaprak durumunda `T` yeni değerle değiştiriliyor, ama eski düğümün `free` edildiği söylenmedi. Yukarıdaki kodda eski düğüm `tmp` ile tutulup serbest bırakılmıştır. Aksi hâlde bellek sızıntısı (memory leak) olur.

Hocanın vurgusu: Ağaçta eleman oluşturma, ekleme, silme ve gezinti kodlarını mutlaka kendiniz yazın ki eliniz alışsın. Mantığını da değişik örneklerle deneyin. Böylece ağaç veri yapısını daha iyi oturtmuş, sindirmiş olursunuz.

13. Dengeleme ve Dengeli Ağaçlar

İkili arama ağacı dejenere olursa, yani patolojik bir ağaca dönüşürse, n elemanlı ağacın yüksekliği n'e kadar çıkabilir. Bildiğimiz bağlantılı liste oluşur. Fakat günlük hayatta ekleme ve silmeler rastgele yapılıyorsa, yeterince uzun süre ekleme ve silme yapıldığında ağacın yüksekliği ortalamada yaklaşık $\log n$ olur. Yani ağaç dengeli olabilir. Ama en kötü durumda yükseklik n'e kadar gider.

Dengeli olmak, bir elemanın $\log n$ mertebesinde, yani $O(\log n)$ karmaşıklıkla aranıp bulunabileceği anlamına gelir. En kötü durumda arama, ekleme ve silme için seviye kadar sorgu yapmak gerekir. Bu yüzden bu işlemler $O(h)$ sürer, burada h yüksekliktir. h'nin en kötü değeri n'dir. Bu, ağacın bağlantılı listeye dönüştüğü durumdur: "buna bak, buraya gel, buna bak, buraya gel" diye bütün düğümler tek tek dolaşılır ve işlem $O(n)$ olur.

Bu engellenebilir. Hocanın ifadesiyle, şimdiye kadar ikili arama ağacında oluşturma, arama, ekleme ve silmeye baktık. Ama bunları çok daha etkin, optimal hâle getiren, yani ağacın $\log n$ mertebesinde kalmasını sağlayan algoritmalar vardır. Bunlara dengeleme politikaları ya da ikili ağaç dengeleme algoritmaları denir:

- AVL ağacı (en kolayı),
- kırmızı-siyah ağaç (red-black tree),
- splay tree,
- B-tree, 2-3 ve 2-3-4 ağaçları.

Hoca AVL'nin fikrini sayısal bir örnekle gösterdi. Ağaçta 4 ve 6 vardır (6, 4'ün sağ çocuğudur). Şimdi 8 eklenecektir. Normal ekleme algoritmasına göre 8, 4'ten büyüktür, sağa gidilir. 6'dan büyüktür, sağa gidilir. `NULL`'a eklenir. Sonuç bir bağlantılı listedir, patolojik bir ağaç oluşmuştur:



AVL ağacı ekleme sırasında bunu kontrol eder ve ağacı değiştirir: 6 kök yapılır, 4 sola, 8 sağa alınır. Ağaç saat yönünün tersine bir tık döndürülmüş olur ve dengelenir. Hoca bunun bir iki özel durumu olduğunu, onların da kontrol edilmesi gerektiğini söyledi. Her eklemede ve her silmede bu tür dönüşler (rotation) yapılarak ağacın yüksekliği $\log n$'de ya da ona en yakın mertebede tutulur.

Ek bilgi (derste anlatılmadı): Bu örnekteki saat yönünün tersine dönüşe sola döndürme (left rotation) denir. AVL ağacında her düğüm için sol ve sağ alt ağaç yükseklikleri arasındaki fark en fazla 1 olabilir. Fark 2'ye çıktığında tek ya da çift döndürme yapılarak denge yeniden sağlanır.

Hocaya göre kırmızı-siyah ağaç bundan biraz daha karışık olsa da yine basit bir algoritmadır. Daha çok boyutlu yapılara ve veri tabanı indeksleri gibi karmaşık yapılara geçildiğinde splay tree ve B-tree'ler çok kullanılır. Günümüzde bunlar veri tabanı yönetim sistemlerinin indeks mekanizmalarında standart olarak kullanılan dengeleme politikalarıdır.

Hoca dengeleme algoritmalarının bu derste ayrıntılı işlenmeyeceğini söyledi. Ama merak edenler için cevabı verdi: Basit ekleme ve silmeyle yetinip rastgeleliğin şansına güvenmek zorunda değiliz. Kırmızı-siyah ağaç ya da AVL ağacı kullanılarak dengeli ikili arama ağaçları oluşturulabilir ve dengeli tutulabilir. Böylece eleman aramaları $\log n$ mertebesinde kalır. Hoca $\log n$ karmaşıklığın ne kadar faydalı olduğunu dönem boyunca sürekli söylediklerini hatırlattı.

Örnek Sorular

Soru 1 (derste sözlü, 00:16:24): Bir ağacın en fazla 3 çocuklu düğümleri olduğu söyleniyor. Düğümlerin bir kısmı yalnız 2 çocukluysa, bu ağaca yine üçlü ağaç mı denir?

> **Çözüm:** Evet. Ağacın kaçlı olduğunu, bir düğümün sahip olduğu en fazla çocuk sayısı (ağacın derecesi) belirler. Hocanın örneğinde düğümlerden 3, 2, 1 çocuk çıkıyordu, ama bir düğümden 4 çocuk çıktığı için ağaç 4'lü ağaçtı.

Soru 2 (derste sözlü, 00:31:34 - 00:33:51): Bölüm 4.7'deki ağaçta (A kök; B ve C, A'nın çocukları; D ve E, B'nin çocukları; F, G ve H, C'nin çocukları; I, H'nin çocuğu) aşağıdakileri bulun: B'nin rolü, C'nin B ile ilişkisi, yapraklar, iç düğümler, E'nin seviyesi, ağacın yüksekliği, B'nin ve ağacın derecesi, I'nın ataları, C'nin alt soyu.

> **Çözüm:** B, D ve E'nin ebeveynidir. C, A'nın çocuğu ve B'nin kardeşidir. Yapraklar D, E, F, G, I'dır. İç düğümler A, B, C, H'dir. E'nin seviyesi 3'tür. Ağacın yüksekliği 3'tür (A → C → H → I). B'nin derecesi 2, ağacın derecesi 3'tür. I'nın ataları H, C, A'dır. C'nin alt soyu F, G, H, I'dır.

Soru 3 (derste sözlü, 01:10:44 - 01:12:10): Kökü A olan 9 düğümlü ağaç (A'nın çocukları B ve C; B'nin çocukları D ve E; C'nin çocukları F ve G; D'nin çocukları H ve I) full mü, complete mi, perfect mi, dengeli mi?

> **Çözüm:** Full'dür (her düğümün 0 ya da 2 çocuğu var). Complete'tir (son seviye hariç seviyeler dolu, son seviye soldan dolu). Perfect değildir (son seviye tam dolu değil). Yükseklik 3'tür ve $\log_2 9 \approx 3{,}17$ olduğu için dengelidir. Derste hoca $\log_2 9$'u yanlış hesapladığı için dengeli olup olmadığı sonuçlanmadı. Doğru sonuç python ile doğrulanmıştır.

Soru 4 (derste sözlü, 01:43:07): Bir ağaç kopyalanırken ziyaret (veri kopyalama) neden en sona konmalıdır?

> **Çözüm:** Hocanın cevabı: Önce ağacın yapısı, yani düğümler oluşturulmalı, sonra veri oraya kopyalanmalıdır. Bu yüzden kopyalama sol, sağ, ziyaret sırasıyla, yani postorder gezintiyle yapılır.

Soru 5 (derste sözlü, 02:00:47): İki ağacın (`first`, `second`) eşit olup olmadığını test etmek için preorder, inorder ve postorder gezintilerinden hangisi daha uygundur?

> **Çözüm:** Preorder. Önce kökteki veriler karşılaştırılır. Eşit değillerse alt ağaçlara inmeden doğrudan "eşit değil" sonucu verilir. Sınıftan önce "inorder" cevabı geldi. Hoca inorder'da önce sola gidildiğini, oysa önce değerin karşılaştırılması gerektiğini hatırlattı ve "preorder" cevabını onayladı. Kod Bölüm 10.2'dedir.

Soru 6 (derste sözlü, 02:11:05 - 02:12:07): Aşağıdaki ağaçlardan hangileri ikili arama ağacıdır?

- Kökü 20 olan, 15, 12 ve 10 değerlerini içeren ağaç.
- Kökü 30 olan ağaç: 30'un solunda 5, sağında 40; 5'in solunda 2.
- Kökü 60 olan ağaç: 60'ın sağında 70; 70'in solunda 65, sağında 80.

> **Çözüm:** Birinci ağaç ikili arama ağacı değildir: 10, bulunduğu yerde 15'ten büyük olması gereken bir alt ağaçtadır. İkinci ağaç dengesizdir ama ikili arama ağacıdır. Üçüncü ağaç ikili arama ağacıdır: 65, hem 60'tan büyük hem 70'ten küçüktür.

Soru 7 (derste sözlü, 02:14:05 - 02:15:58): Kökü 44 olan ikili arama ağacında 25 ve 76 aranıyor. Hangi düğümlerden geçilir, sonuç nedir?

> **Çözüm:** 25 için yol 44 → 17 → 32 → 28 → `NULL` olur. 25 ağaçta yoktur. 76 için yol 44 → 88 → 65 → 82 → 76 olur. 76 bulunur.

Soru 8 (derste sözlü, 02:17:09 - 02:23:44): Kökü 94 olan ağaçta (Bölüm 12.5) sırasıyla yaprak 5, tek çocuklu 24 ve iki çocuklu 10 silinirse (her biri ayrı ayrı, başlangıç ağacından) ne olur?

> **Çözüm:** 5 doğrudan silinir. 24 silinince çocuğu 11 (alt soyu 17 ile birlikte) 10'un sağına bağlanır. 10 silinirken sağ alt ağacın en küçüğü 11 yukarı kopyalanır ve eski 11'in yerine sağ çocuğu 17 bağlanır; sonuç preorder 94, 11, 5, 24, 17. Diğer seçenekte sol alt ağacın en büyüğü 5 yukarı kopyalanır, 11 yerinde kalır.

Dersin Toparlaması

Bu derste ağaç veri yapısı temelden ele alındı. Ağaç, kapalı çevrim içermeyen, kökten her düğüme tek yolla erişilen sonlu bir düğüm kümesidir. Kök, ebeveyn, çocuk, yaprak, kardeş, derece, yükseklik, derinlik, seviye ve alt ağaç kavramları örnekler üzerinde tanımlandı. Kardeşlerin derinliğinin aynı, yüksekliklerinin ise alt soylarına göre farklı olabileceği gösterildi. Ağacı dizi ile tutmanın, özellikle dejenere ağaçlarda 2^n mertebesinde bellek israfına yol açtığı görüldü. Bu yüzden düğümlerin sol ve sağ işaretçilerle bağlandığı bağlantılı liste gösterimine geçildi. İkili ağacın full, complete, perfect, dengeli ve dejenere türleri karşılaştırıldı.

Dersin ikinci yarısında ağaçta gezinti işlendi. Derinlik öncelikli preorder, inorder ve postorder gezintileri aynı ağaç üzerinde adım adım yürütüldü. Inorder'ın yığınla özyinelemesiz biçimi ve kuyrukla genişlik öncelikli (seviye sırası) gezinti gösterildi. Kopyalama için postorder, eşitlik kontrolü için preorder seçildi. Çok çocuklu ağaçların ilk çocuk ve ilk kardeş işaretçileriyle ikili ağaç olarak tutulabileceği, böylece ikili ağaç fonksiyonlarının bütün ağaçlarda kullanılabileceği gösterildi. Son olarak ikili arama ağacında arama, ekleme ve üç durumlu silme anlatıldı ve bu işlemlerin $O(h)$ sürdüğü, ağaç dejenere olursa $O(n)$'e çıktığı vurgulandı. AVL, kırmızı-siyah, splay ve B-tree gibi dengeleme yaklaşımları, yüksekliği $O(\log n)$ civarında tutmanın yolu olarak tanıtıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.